Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

Отчет

по лабораторной работе №4

по дисциплине «Программирование на языке Java»

на тему: «Работа с файлами»

Вариант 3

Выполнили студенты группы 23ВВВ3:

Балаев Г. С.

Саранцев. Е. А.

Приняли:

Юрова О. В.

Карамышева Н. С.

Пенза, 2026

Цель работы

изучить работу с файлами и механизмы сериализации данных.

Лабораторное задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав сохранение в файл и загрузку данных из файла. Предусмотреть сохранение данных, как в текстовом виде, так и в двоичном (с использованием механизма сериализации). Для этого нужно добавить 4 кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно. Кроме того, в программе нужно предусмотреть использование стандартного диалога открытия файла (JFileChooser).

Ход работы:

1. Открыли прошлый проект в IDE JetBrains и создали новый класс InvalidDataException.

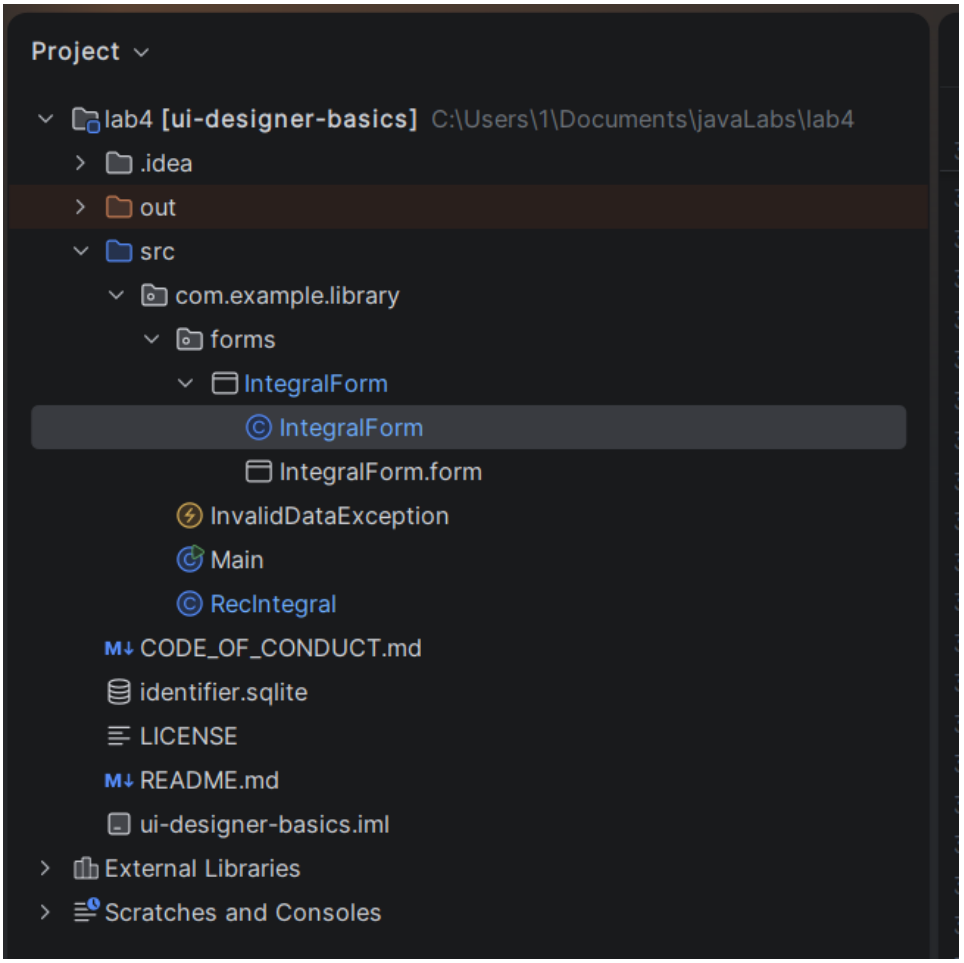

Рисунок 1 – Проект.

2. Добавили сохранение/загрузку в bin, text, json.

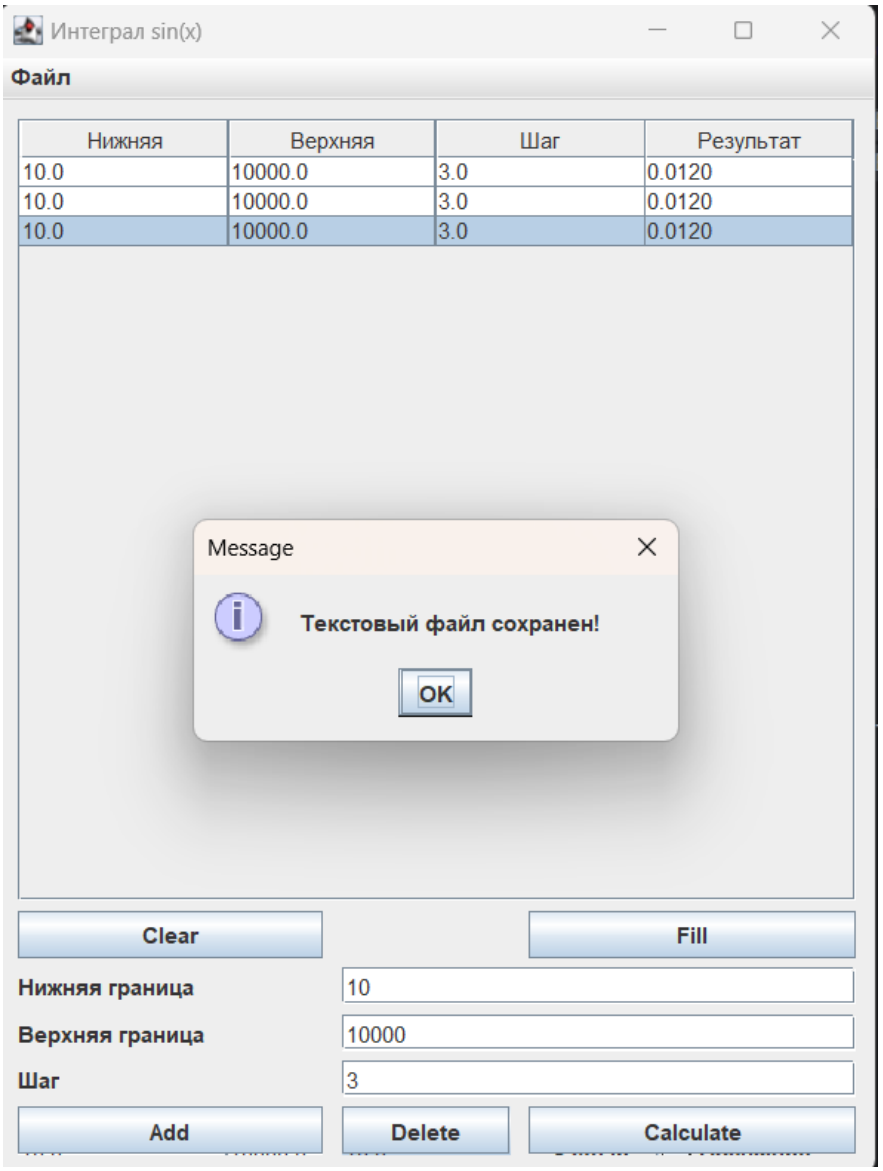

Рисунок 2 – Сохранение text.

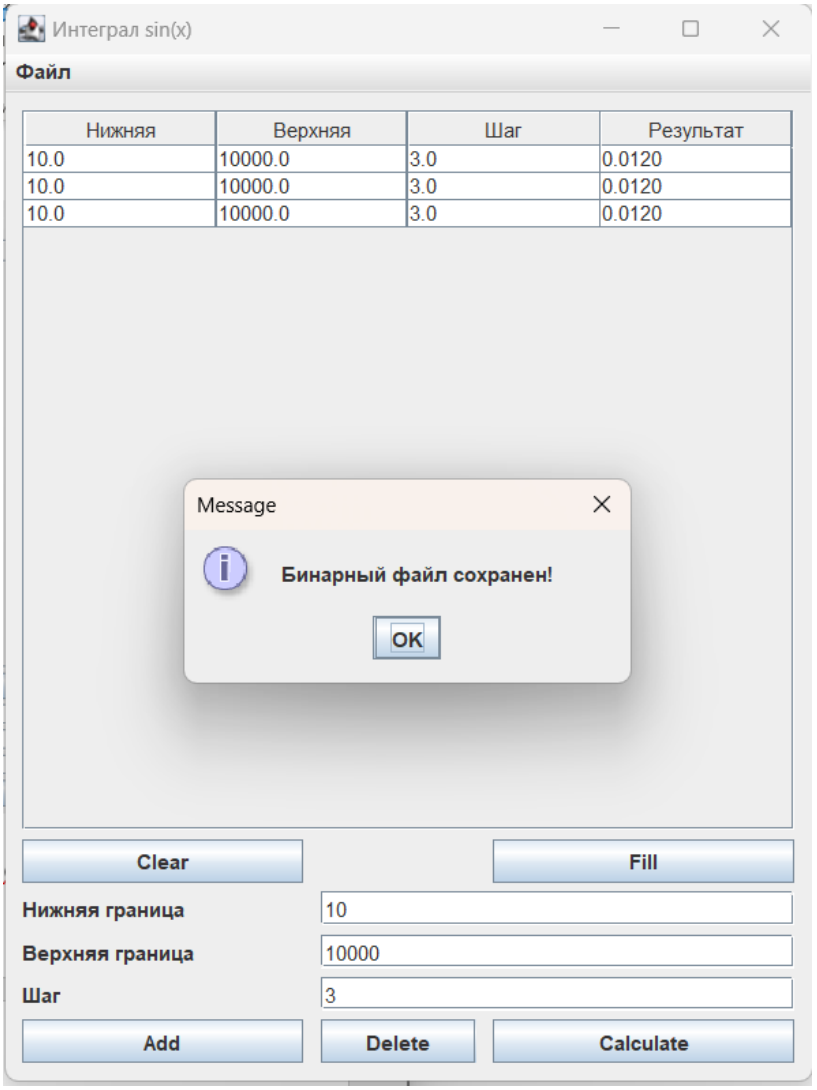

Рисунок 3 – Сохранение bin.

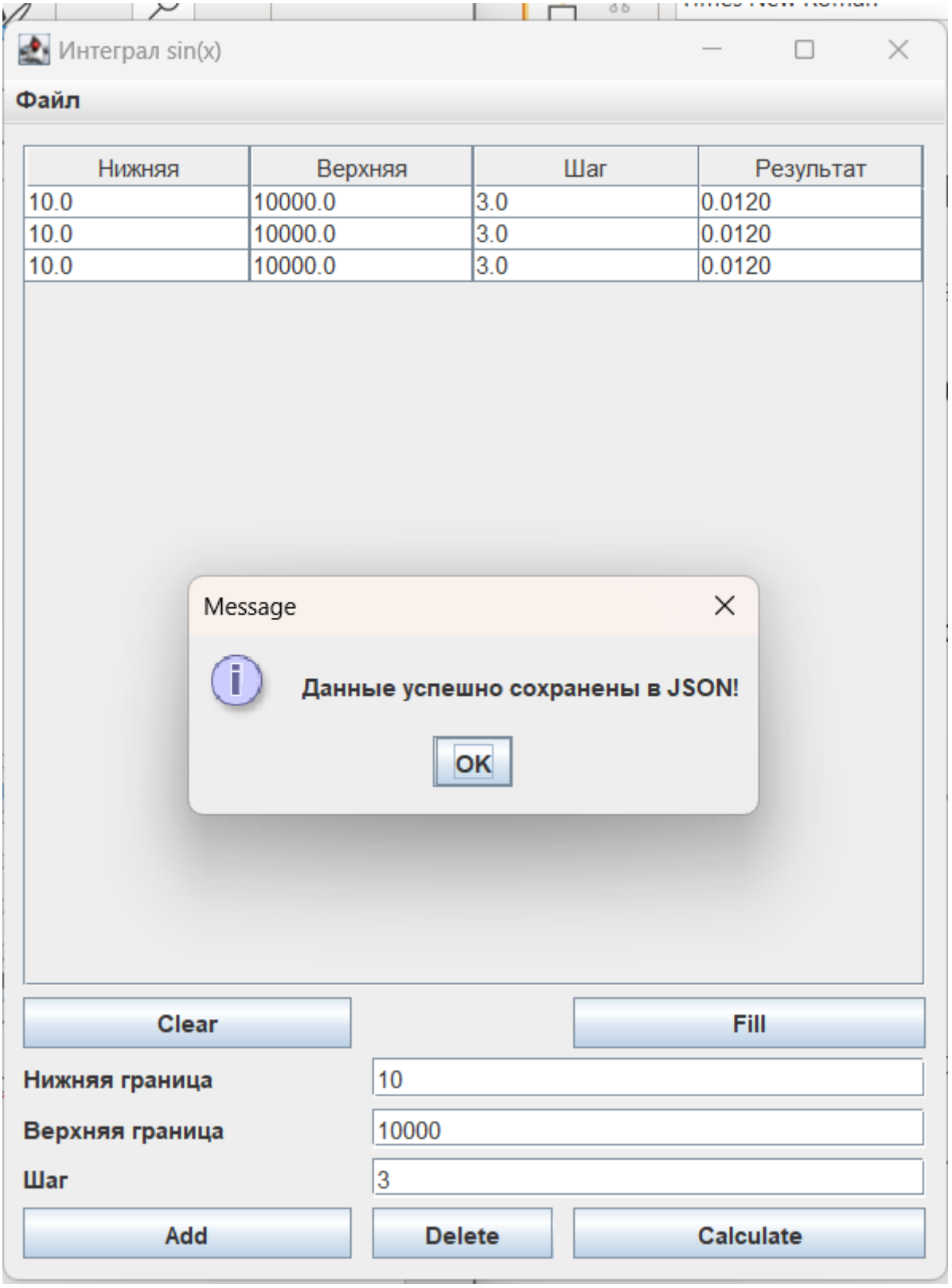

Рисунок 4 – Сохранение JSON.

Результат работы программы:

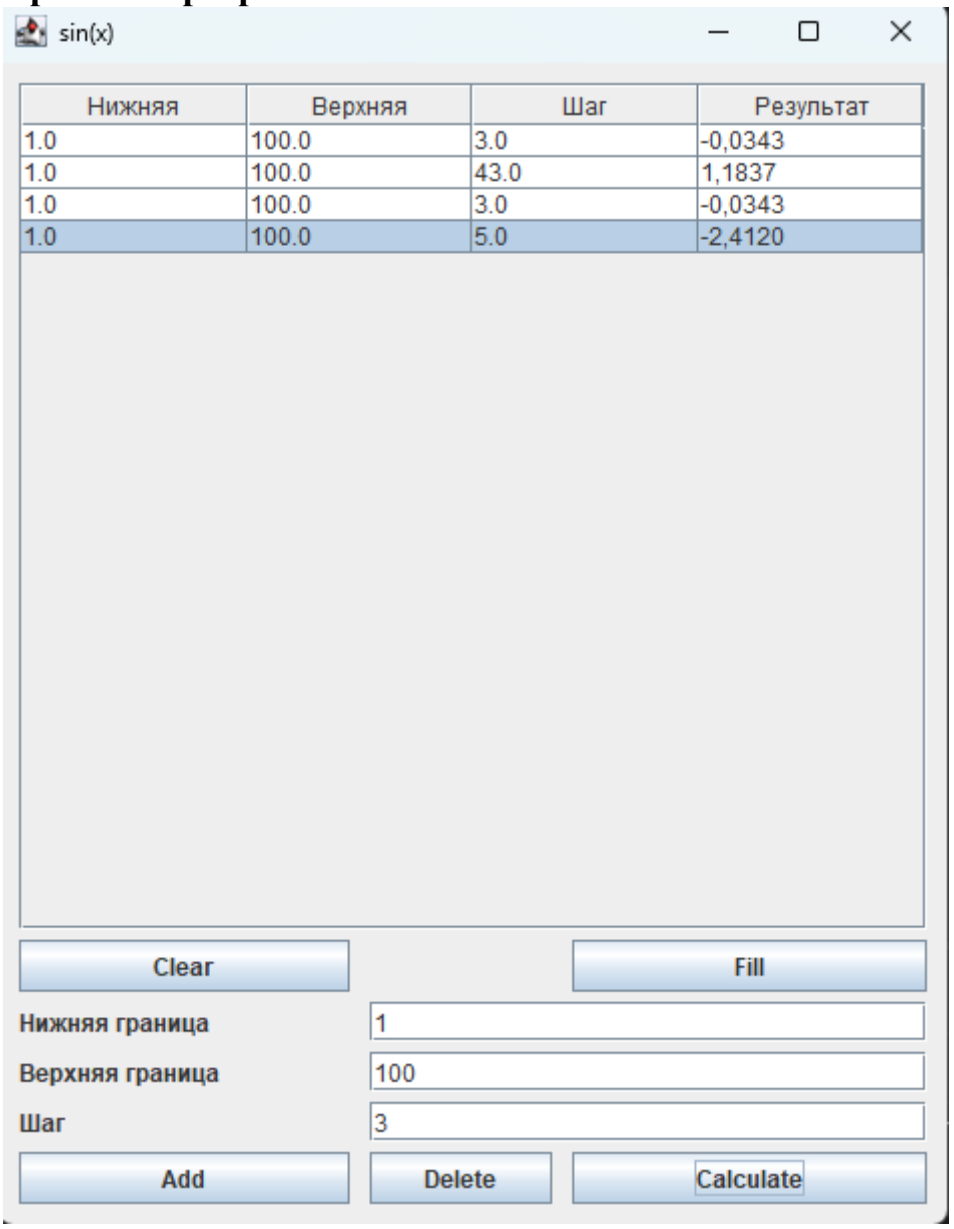

Рисунок 5 – Результат работы программы.

Вывод:

В ходе данной лабораторной работы мы научились работе с файлы.

Листинг: Main.java:

```
package com.example.library;

import javax.swing.*;
import com.example.library.forms.IntegralForm;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Интеграл sin(x)");
            IntegralForm form = new IntegralForm();

            JMenuBar menuBar = new JMenuBar();

            JMenu fileMenu = new JMenu("Файл");

            JMenuItem itemSaveText = new JMenuItem("Сохранить в текст");
            JMenuItem itemLoadText = new JMenuItem("Загрузить из
текста");
            JMenuItem itemSaveBin = new JMenuItem("Сохранить
(бинарно)");
            JMenuItem itemLoadBin = new JMenuItem("Загрузить
(бинарно)");
            JMenuItem itemSaveJson = new JMenuItem("Сохранить (json)");
            JMenuItem itemLoadJson = new JMenuItem("Загрузить (json)");

            itemSaveText.addActionListener(e -> form.saveAsText());
            itemLoadText.addActionListener(e -> form.loadFromText());
            itemSaveBin.addActionListener(e -> form.saveAsBinary());
            itemLoadBin.addActionListener(e -> form.loadFromBinary());
            itemSaveJson.addActionListener(e -> form.saveAsJson());
            itemLoadJson.addActionListener(e -> form.loadFromJson());

            fileMenu.add(itemSaveText);
            fileMenu.add(itemLoadText);
            fileMenu.addSeparator();
            fileMenu.add(itemSaveBin);
            fileMenu.add(itemLoadBin);
            fileMenu.add(itemSaveJson);
            fileMenu.add(itemLoadJson);

            menuBar.add(fileMenu);

            frame.setJMenuBar(menuBar);

            frame.setContentPane(form.getRootPanel());
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.pack();
```

```
            frame.setLocationRelativeTo(null);
            frame.setVisible(true);
        });
    }
}
```

IntegralForm.java:

```
package com.example.library.forms;

import com.example.library.InvalidDataException;
import com.example.library.RecIntegral;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.nio.file.Files;
import java.util.ArrayList;
import java.util.List;
import java.io.*;
import java.util.Locale;
import java.util.stream.Collectors;

public class IntegralForm {
    private JPanel rootPanel;
    private JTextField fieldLowerBound;
    private JTextField fieldUpperBound;
    private JTextField fieldStep;
    private JButton btnAdd;
    private JButton btnDelete;
    private JButton btnCalculate;
    private JTable table1;
    private JButton btnClear;
    private JButton btnFill;

    private DefaultTableModel model;
    private List<RecIntegral> listRecords = new ArrayList<>();

    public IntegralForm() {
        String[] columns = {"Нижняя", "Верхняя", "Шаг", "Результат"};
        model = new DefaultTableModel(columns, 0) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return column != 3;
            }

            @Override
            public void setValueAt(Object aValue, int row, int column) {
                if (column == 3) {
                    super.setValueAt(aValue, row, column);
                    return;
                }
                try {
                    double val =
Double.parseDouble(aValue.toString().replace(',', '.'));
                    RecIntegral rec = listRecords.get(row);

                    double high = rec.getUpperBound();
                    double low = rec.getLowerBound();
                    double step = rec.getStep();

                    if (val < 0.000001 || val > 1000000) {
                        throw new InvalidDataException("Число " + val + " вне
диапазона");
                    }
                    if (column == 0) {
                        if (high - val < 0) throw new
InvalidDataException("Нижняя > верхней");
                        if (high - val < step) throw new
InvalidDataException("Шаг больше интервала");
```

```
                    }
                    if (column == 1) {
                        if (val - low < 0) throw new
InvalidDataException("Верхняя < нижней");
                        if (val - low < step) throw new
InvalidDataException("Шаг больше интервала");
                    }
                    if (column == 2) {
                        if (high - low < val) throw new
InvalidDataException("Шаг больше интервала");
                    }

                    super.setValueAt(aValue, row, column);

                    if (row >= 0 && row < listRecords.size()) {
                        if (column == 0) rec.setLowerBound(val);
                        else if (column == 1) rec.setUpperBound(val);
                        else if (column == 2) rec.setStep(val);

                        rec.setResult(0);
                        super.setValueAt("", row, 3);
                    }
                } catch (NumberFormatException e) {
                    JOptionPane.showMessageDialog(rootPanel, "Введите число",
"Ошибка", JOptionPane.ERROR_MESSAGE);
                } catch (InvalidDataException e) {
                    JOptionPane.showMessageDialog(rootPanel, e.getMessage(),
"Ошибка", JOptionPane.WARNING_MESSAGE);
                }
            }
        };
        table1.setModel(model);

        btnAdd.addActionListener(e -> {
            try {
                double low =
Double.parseDouble(fieldLowerBound.getText().replace(',', '.'));
                double high =
Double.parseDouble(fieldUpperBound.getText().replace(',', '.'));
                double step =
Double.parseDouble(fieldStep.getText().replace(',', '.'));

                if (high - low < 0) throw new InvalidDataException("Нижняя
граница > Верхней");
                if (high - low < step) throw new InvalidDataException("Шаг
больше интервала");

                RecIntegral record = new RecIntegral(low, high, step, 0);
                listRecords.add(record);
                model.addRow(new Object[]{low, high, step, ""});
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(rootPanel, "Ошибка: " +
ex.getMessage());
            }
        });

        btnCalculate.addActionListener(e -> {
            try {
                int row = table1.getSelectedRow();
                if (row == -1) throw new Exception("Выберите строку");

                double left = Double.parseDouble(model.getValueAt(row,
0).toString());
                double right = Double.parseDouble(model.getValueAt(row,
1).toString());
                double step = Double.parseDouble(model.getValueAt(row,
2).toString());
```

```
                double res = RecIntegral.calculate(left, right, step);
                listRecords.get(row).setResult(res);
                model.setValueAt(String.format(Locale.US, "%.4f", res), row, 3);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(rootPanel, ex.getMessage());
            }
        });

        btnDelete.addActionListener(e -> {
            int row = table1.getSelectedRow();
            if (row != -1) {
                listRecords.remove(row);
                model.removeRow(row);
            }
        });

        btnClear.addActionListener(e -> model.setRowCount(0));
        btnFill.addActionListener(e -> {
            model.setRowCount(0);
            for (RecIntegral rec : listRecords) {
                String resStr = (rec.getResult() == 0) ? "" :
String.format(Locale.US, "%.4f", rec.getResult());
                model.addRow(new Object[]{rec.getLowerBound(),
rec.getUpperBound(), rec.getStep(), resStr});
            }
        });
    }


    public void saveAsText() {
        List<RecIntegral> toSave =
listRecords.stream().collect(Collectors.toList());

        if (toSave.isEmpty()) {
            JOptionPane.showMessageDialog(rootPanel, "Нет вычисленных данных для
сохранения!");
            return;
        }

        JFileChooser fc = new JFileChooser();
        if (fc.showSaveDialog(rootPanel) == JFileChooser.APPROVE_OPTION) {
            try (FileWriter fw = new FileWriter(fc.getSelectedFile())) {
                for (RecIntegral rec : toSave) {
                    fw.write(String.format(Locale.US, "%f;%f;%f;%f\n",
                            rec.getLowerBound(), rec.getUpperBound(),
rec.getStep(), rec.getResult()));
                }
                JOptionPane.showMessageDialog(rootPanel, "Текстовый файл
сохранен!");
            } catch (IOException e) {
                JOptionPane.showMessageDialog(rootPanel, "Ошибка записи: " +
e.getMessage());
            }
        }
    }

    public void loadFromText() {
        JFileChooser fc = new JFileChooser();
        if (fc.showOpenDialog(rootPanel) == JFileChooser.APPROVE_OPTION) {
            try (FileReader fr = new FileReader(fc.getSelectedFile());
                 BufferedReader br = new BufferedReader(fr)) {

                listRecords.clear();
                model.setRowCount(0);
                String line;
                while ((line = br.readLine()) != null) {
                    String[] p = line.split(";");
                    if (p.length == 4) {
```

```
                        double low = Double.parseDouble(p[0]);
                        double high = Double.parseDouble(p[1]);
                        double step = Double.parseDouble(p[2]);
                        double res = Double.parseDouble(p[3]);

                        RecIntegral rec = new RecIntegral(low, high, step, res);
                        listRecords.add(rec);
                        model.addRow(new Object[]{low, high, step,
String.format(Locale.US, "%.4f", res)});
                    }
                }
            } catch (Exception e) {
                JOptionPane.showMessageDialog(rootPanel, "Ошибка чтения: " +
e.getMessage());
            }
        }
    }

    public void saveAsBinary() {
        List<RecIntegral> toSave = listRecords.stream()
                .collect(Collectors.toList());

        if (toSave.isEmpty()) {
            JOptionPane.showMessageDialog(rootPanel, "Нет данных для
сохранения!");
            return;
        }

        JFileChooser fc = new JFileChooser();
        if (fc.showSaveDialog(rootPanel) == JFileChooser.APPROVE_OPTION) {
            try (ObjectOutputStream out = new ObjectOutputStream(
                    new BufferedOutputStream(new
FileOutputStream(fc.getSelectedFile()))) {
                out.writeObject(toSave);
                JOptionPane.showMessageDialog(rootPanel, "Бинарный файл
сохранен!");
            } catch (IOException e) {
                JOptionPane.showMessageDialog(rootPanel, "Ошибка сериализации: "
+ e.getMessage());
            }
        }
    }

    public void loadFromBinary() {
        JFileChooser fc = new JFileChooser();
        if (fc.showOpenDialog(rootPanel) == JFileChooser.APPROVE_OPTION) {
            try (ObjectInputStream in = new ObjectInputStream(
                    new BufferedInputStream(new
FileInputStream(fc.getSelectedFile()))) {

                @SuppressWarnings("unchecked")
                List<RecIntegral> loaded = (List<RecIntegral>) in.readObject();

                listRecords.clear();
                listRecords.addAll(loaded);

                model.setRowCount(0);
                for (RecIntegral rec : listRecords) {
                    model.addRow(new Object[]{rec.getLowerBound(),
rec.getUpperBound(), rec.getStep(),
                            String.format(Locale.US, "%.4f", rec.getResult())});
                }
            } catch (Exception e) {
                JOptionPane.showMessageDialog(rootPanel, "Ошибка десериализации:
" + e.getMessage());
            }
        }
    }
```

```
    public void saveAsJson() {
        // Сохраняем только те записи, которые были вычислены (результат != 0)
        List<RecIntegral> toSave =
listRecords.stream().collect(Collectors.toList());

        if (toSave.isEmpty()) {
            JOptionPane.showMessageDialog(rootPanel, "Нет вычисленных данных для
сохранения!");
            return;
        }

        JFileChooser fc = new JFileChooser();
        if (fc.showSaveDialog(rootPanel) == JFileChooser.APPROVE_OPTION) {
            try (PrintWriter out = new PrintWriter(fc.getSelectedFile())) {
                StringBuilder sb = new StringBuilder();
                sb.append("[\n"); // Начало массива

                for (int i = 0; i < toSave.size(); i++) {
                    RecIntegral rec = toSave.get(i);
                    sb.append("  {\n");
                    sb.append(String.format(Locale.US, "    \"lowerBound\":
%f,\n", rec.getLowerBound()));
                    sb.append(String.format(Locale.US, "    \"upperBound\":
%f,\n", rec.getUpperBound()));
                    sb.append(String.format(Locale.US, "    \"step\": %f,\n",
rec.getStep()));
                    sb.append(String.format(Locale.US, "    \"result\": %f\n",
rec.getResult()));
                    sb.append("  }");

                    if (i < toSave.size() - 1) {
                        sb.append(",");
                    }
                    sb.append("\n");
                }
                sb.append("]");

                out.print(sb.toString());
                JOptionPane.showMessageDialog(rootPanel, "Данные успешно
сохранены в JSON!");
            } catch (Exception e) {
                JOptionPane.showMessageDialog(rootPanel, "Ошибка сохранения: " +
e.getMessage());
            }
        }
    }

    public void loadFromJson() {
        JFileChooser fc = new JFileChooser();
        if (fc.showOpenDialog(rootPanel) == JFileChooser.APPROVE_OPTION) {
            try {
                String content = new
String(Files.readAllBytes(fc.getSelectedFile().toPath()));

                content = content.trim();
                if (content.startsWith("[")) content = content.substring(1);
                if (content.endsWith("]")) content = content.substring(0,
content.length() - 1);
                content = content.trim();

                if (content.isEmpty()) {
                    JOptionPane.showMessageDialog(rootPanel, "Файл пуст");
                    return;
                }

                listRecords.clear();
```

```
                model.setRowCount(0);

                String[] objects = content.split("(?<=\\}),");

                for (String objStr : objects) {
                    objStr = objStr.trim();
                    if (objStr.isEmpty()) continue;

                    double low = extractJsonValue(objStr, "lowerBound");
                    double high = extractJsonValue(objStr, "upperBound");
                    double step = extractJsonValue(objStr, "step");
                    double res = extractJsonValue(objStr, "result");

                    RecIntegral rec = new RecIntegral(low, high, step, res);
                    listRecords.add(rec);

                    String resStr = (res == 0) ? "" : String.format(Locale.US,
"%.4f", res);
                    model.addRow(new Object[]{low, high, step, resStr});
                }

                JOptionPane.showMessageDialog(rootPanel, "Данные успешно
загружены из JSON!");

            } catch (Exception e) {
                JOptionPane.showMessageDialog(rootPanel, "Ошибка при разборе
JSON: " + e.getMessage());
            }
        }
    }

    private double extractJsonValue(String json, String key) {

        String searchKey = "\"" + key + "\":";
        int startPos = json.indexOf(searchKey);

        if (startPos == -1) return 0;

        startPos += searchKey.length();

        int endPos = json.indexOf(",", startPos);
        if (endPos == -1) {
            endPos = json.indexOf("}", startPos);
        }
        if (endPos == -1) {
            endPos = json.length();
        }

        String value = json.substring(startPos, endPos).trim();

        value = value.replace("\"", "");

        try {
            return Double.parseDouble(value);
        } catch (NumberFormatException e) {
            return 0;
        }
    }



    public JPanel getRootPanel() {
        return rootPanel;
    }

}
```

RecIntegral.java:

```
package com.example.library;


import static java.lang.Math.sin;


import java.io.*;


public class RecIntegral implements Externalizable  {

    private double lowerBound;

    private double upperBound;

    private double step;

    private double result;


    public RecIntegral() {}


    public RecIntegral(double lowerBound, double upperBound, double
step, double result) throws InvalidDataException {

        checkValue(lowerBound);

        checkValue(upperBound);

        checkValue(step);


        this.lowerBound = lowerBound;

        this.upperBound = upperBound;

        this.step = step;

        this.result = result;

    }


    @Override

    public void writeExternal(ObjectOutput out) throws IOException {

        out.writeDouble(lowerBound);

        out.writeDouble(upperBound);

        out.writeDouble(step);

        out.writeDouble(result);

    }
```

```
    @Override
    public void readExternal(ObjectInput in) throws IOException {
        this.lowerBound = in.readDouble();
        this.upperBound = in.readDouble();
        this.step = in.readDouble();
        this.result = in.readDouble();
    }

    private void checkValue(double val) throws InvalidDataException {
        if (val < 0.000001 || val > 1000000) {
            throw new InvalidDataException("Значение " + val + " вне
диапазона [0.000001, 1000000]");
        }
    }

    public double getLowerBound() { return lowerBound; }
    public void setLowerBound(double lowerBound) { this.lowerBound =
lowerBound; }

    public double getUpperBound() { return upperBound; }
    public void setUpperBound(double upperBound) { this.upperBound =
upperBound; }

    public double getStep() { return step; }
    public void setStep(double step) { this.step = step; }

    public double getResult() { return result; }
    public void setResult(double result) { this.result = result; }

    private static double f(double x) {
        return sin(x);
    }

    public static double calculate(double left, double right, double
step) {
        if (step <= 0) return 0;
        int n = (int)((right - left) / step);
```

```
        if (n <= 0) return 0;

        double result = 0;
        for (int i = 0; i < n; i++) {
            double x0 = left + i * step;
            double x1 = x0 + step;
            if (x1 > right) x1 = right;
            result += (f(x0) + f(x1)) * (x1 - x0) / 2;
        }
        return result;
    }
```

}**InvalidDataException.java:**

```
package com.example.library;

public class InvalidDataException extends Exception {
    public InvalidDataException(String message) {
        super(message);
    }
}
```